

BÁO CÁO BÀI TẬP LỚN: CHƯƠNG TRÌNH CỜ CARO AI

Họ và tên: NGUYỄN TIẾN PHƯƠNG

MSSV: 24022430

1. Mô tả bài toán cờ Caro

- Giới thiệu:** Bài toán yêu cầu xây dựng chương trình chơi cờ Caro đối kháng giữa người (quân X) và máy tính (quân O). Máy tính sử dụng thuật toán tìm kiếm có đối thủ (Adversarial Search) để phân tích và lựa chọn nước đi tối ưu.
- Luật chơi và điều kiện thắng:**
 - Bàn cờ được thiết lập với kích thước 9x9.
 - Hai bên thay phiên nhau đánh quân vào các ô trống trên bàn cờ.
 - Điều kiện thắng:** Người chơi hoặc máy tính sẽ chiến thắng nếu tạo được chuỗi **4 quân liên tiếp** theo chiều ngang, dọc hoặc đường chéo.
 - Đặc tả luật:** Không xét luật chặn hai đầu. Trò chơi kết thúc với kết quả hòa nếu bàn cờ đầy mà không có ai đạt 4 quân liên tiếp.

2. Thiết kế hệ thống và Cấu trúc dữ liệu

- Biểu diễn trạng thái bàn cờ:** Bàn cờ được biểu diễn dưới dạng một mảng hai chiều (ma trận) 9x9. Các phần tử nhận một trong ba trạng thái: . (ô trống), X (người chơi), và O (máy tính).
- Sinh nước đi hợp lệ:** Để giải quyết vấn đề bùng nổ tổ hợp của không gian tìm kiếm, thuật toán không duyệt toàn bộ các ô trống. Thay vào đó, chương trình giới hạn danh sách nước đi cần xét bằng cách chỉ sinh ra các ô trống nằm trong **bán kính 2 ô lân cận** xung quanh các quân cờ đã được đánh. Ở nước đi đầu tiên của ván cờ, AI mặc định đánh vào vị trí trung tâm.
- Kiểm tra trạng thái kết thúc:** Hàm kiểm tra chiến thắng sẽ duyệt từ vị trí quân cờ vừa được đánh, đếm số lượng quân đồng chất liên tiếp theo 4 hướng (ngang, dọc, chéo xuôi, chéo ngược). Nếu bàn cờ không còn ô trống nào (**is_board_full**), hàm trả về trạng thái hòa.

3. Thuật toán tìm kiếm và Hàm đánh giá

3.1. Thuật toán Minimax và Alpha-Beta Pruning

- Thuật toán Minimax:** Được cài đặt để xử lý bài toán trò chơi hai người, tổng bằng không. AI (MAX) cố gắng tối đa hóa điểm số, trong khi Người chơi (MIN) giả định sẽ chọn nước đi giảm thiểu điểm của AI. Thuật toán có sử dụng giới hạn độ sâu (Depth Limit) để ngắt việc tìm kiếm nhánh đệ quy.

- **Thuật toán Alpha-Beta Pruning:** Là bản nâng cấp của Minimax giúp cắt bỏ các nhánh không cần thiết. Thuật toán duy trì hai biến: **alpha** (giá trị tốt nhất của MAX) và **beta** (giá trị tốt nhất của MIN). Trong quá trình duyệt cây, nếu xuất hiện điều kiện **beta <= alpha**, thuật toán lập tức cắt nhánh (break) vì nhánh đó không thể mang lại kết quả tốt hơn.

3.2. Hàm đánh giá trạng thái (Heuristic)

Do không thể duyệt đến cuối trò chơi, hàm đánh giá trạng thái được gọi khi thuật toán đạt giới hạn độ sâu. Hàm hoạt động bằng cách trừ các "cửa sổ 4 ô" trên bàn cờ và cộng/trừ điểm.

- **Ưu tiên tấn công:** +100.000 điểm cho chuỗi 4 quân O; +5.000 điểm cho chuỗi 3 quân O thoáng; +200 điểm cho chuỗi 2 quân O thoáng.
- **Phòng ngự sống còn:** Phạt rất nặng (-50.000 điểm) nếu phát hiện chuỗi 3 quân X của người chơi, qua đó ép AI bằng mọi giá phải chọn nước đi để chặn đầu.

4. Kết quả thực nghiệm và Nhận xét

Để đánh giá hiệu quả thuật toán, chương trình được chạy thực nghiệm trên 6 trạng thái bàn cờ khác nhau. Dưới đây là bảng dữ liệu phân tích tập trung tại **Độ sâu (Depth) = 3**, sử dụng cùng một hàm đánh giá cho cả hai thuật toán:

ID	Mô tả Trạng thái kiểm thử	Thuật toán Minimax (Độ sâu 3)	Thuật toán Alpha-Beta (Độ sâu 3)	Điểm số & Nước đi (O)
1	Trạng thái đầu ván	- Số nút đã xét: 33.865 - Thời gian: 4.665 ms	- Số nút đã xét: 4.803 - Thời gian: 645 ms	- Điểm: +3.200 - Ô chọn: (2, 4)
2	Trạng thái giữa ván	- Số nút đã xét: 105.363	- Số nút đã xét: 7.389	- Điểm: -100.001

		- Thời gian: 25.562 ms	- Thời gian: 2.065 ms	- Ô chọn: (0, 1)
3	Máy có thể thắng ngay	- Số nốt đã xét: 66.776 - Thời gian: 17.279 ms	- Số nốt đã xét: 1.952 - Thời gian: 475 ms	- Điểm: +100.002 - Ô chọn: (3, 1)
4	Người sắp thắng, máy cần chặn	- Số nốt đã xét: 67.081 - Thời gian: 17.808 ms	- Số nốt đã xét: 12.431 - Thời gian: 3.251 ms	- Điểm: -50.000 - Ô chọn: (1, 0)
5	Hai bên đều có cơ hội tấn công	- Số nốt đã xét: 84.954 - Thời gian: 15.082 ms	- Số nốt đã xét: 13.853 - Thời gian: 1.939 ms	- Điểm: +5.200 - Ô chọn: (2, 4)
6	Nhiều nhánh (các quân rải rác)	- Số nốt đã xét: 284.510	- Số nốt đã xét: 29.412	- Điểm: +400

		- Thời gian: 49.120 ms	- Thời gian: 4.215 ms	- Ô chọn: (2, 6)
--	--	---------------------------	--------------------------	------------------

4.1. Nhận xét phân tích thuật toán

- **Về tính đồng nhất:** Thuật toán Alpha-Beta luôn trả về cùng một nước đi và điểm số y hệt như Minimax. Điều này chứng minh quá trình cắt nhánh Alpha-Beta diễn ra chính xác mà không làm sai lệch kết quả tìm kiếm tối ưu.
- **Về hiệu năng tối ưu trạng thái:** Alpha-Beta giảm được một lượng trạng thái khổng lồ so với Minimax (giảm trung bình từ 80% đến 95%). Đặc biệt ở trạng thái "Máy có thể thắng ngay" (Trạng thái 3), Alpha-Beta sớm tìm thấy nước đi điểm cao và cắt nhánh lập tức, giảm từ gần 67.000 nốt xuống còn dưới 2.000 nốt.
- **Về ảnh hưởng của độ sâu (Depth):** Khi tăng độ sâu lên 3, thời gian chạy của Minimax tăng theo hàm mũ (lên tới 49 giây ở trạng thái phức tạp), không phù hợp để chơi thực tế. Ngược lại, Alpha-Beta duy trì thời gian tính toán dao động từ 0.5s đến 4s. Độ sâu càng cao, AI phân tích càng xa, biết giăng bẫy chuỗi thay vì chỉ phản ứng "ăn xối" cục bộ như ở độ sâu 1.

4.2. Nhận xét hàm đánh giá và định hướng cải tiến

- **Ưu điểm & Hạn chế của Hàm đánh giá:** Hàm Heuristic hiện tại xử lý tính toán nhanh chóng, AI nhận biết rất tốt các thế cờ nguy hiểm cơ bản và biết ưu tiên chặn chuỗi 3 của người chơi. Tuy nhiên, hạn chế là hàm chỉ nhìn các ô liền kề, chưa hiểu được các nước đi chiến lược mang tính dọn đường xa (ví dụ: tạo cấu trúc "bẫy xiên đôi" hoặc các chuỗi bị cắt đứt ở giữa).
- **Hướng cải tiến tương lai:** Để nâng cao trình độ AI, nếu tiếp tục phát triển, em sẽ tối ưu thuật toán cắt nhánh bằng kỹ thuật *Move Ordering* (sắp xếp nước đi tiềm năng lên trước) và sử dụng bảng băm *Zobrist Hashing / Transposition Table* để ghi nhớ điểm của các thế cờ đã xét qua, tránh tính toán lại.

5. Tài liệu tham khảo

1. Tài liệu môn học Cơ sở Trí tuệ Nhân tạo - Trường Đại học Công nghệ.
2. Tham khảo logic tổ chức dự án và giao diện từ: Repository [gomokuAI-py](https://github.com/husus/gomokuAI-py) (github.com/husus/gomokuAI-py).
3. Tham khảo cách phân tích benchmark và thiết kế cấu hình từ: Repository [Caro_AI](https://github.com/MonHauVD/Caro_AI) (github.com/MonHauVD/Caro_AI).